

Scaling the Swiss National ABM: A High-Performance, Native Java Implementation for generating daily plans in MATSim

Joschka Bischoff
MATSim User Meeting 2026



Outline.

1.

Getting the Problem:
Why we need this

2.

Into the details.

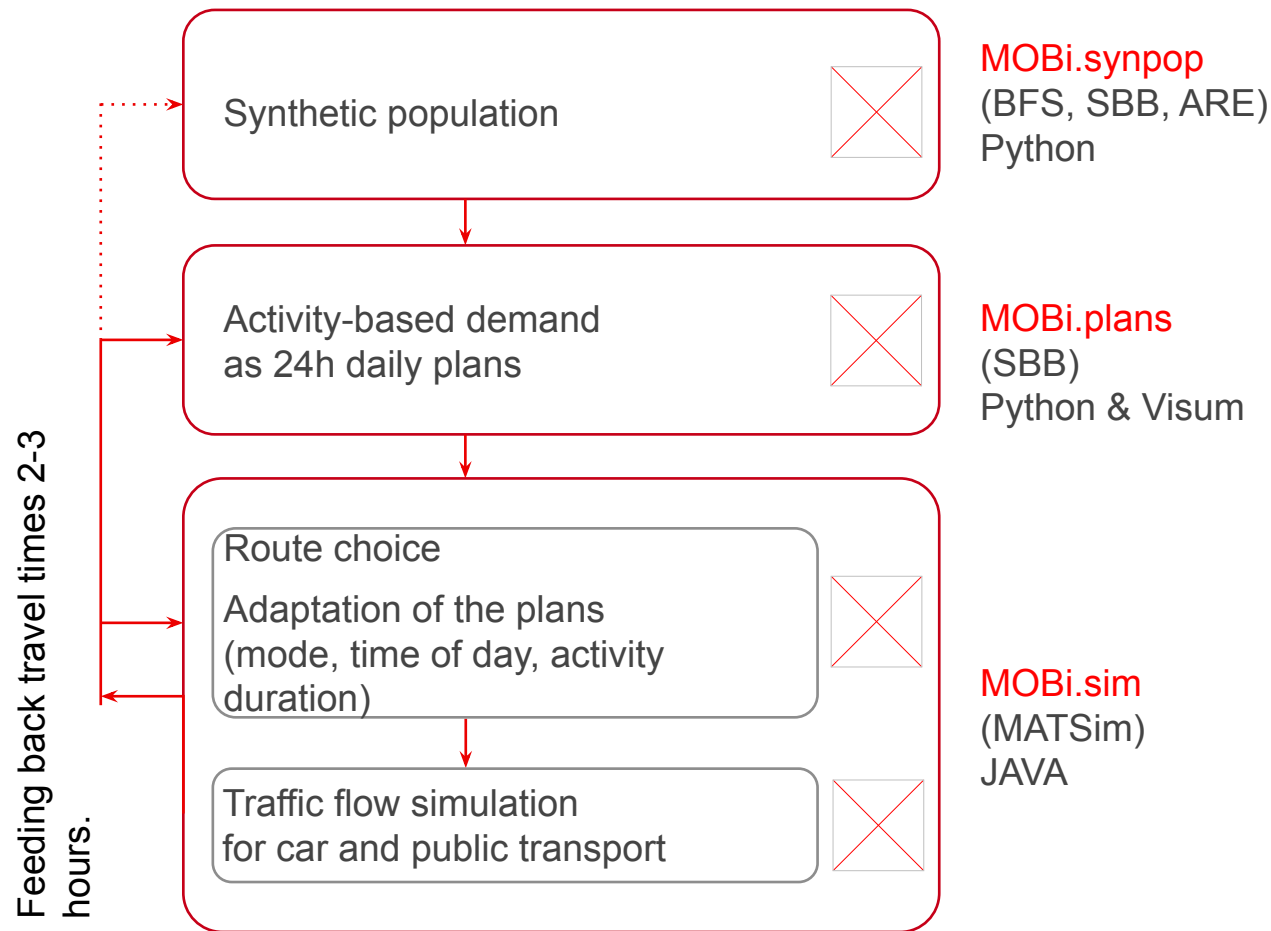
3.

Great.
Now can I use this?



1. Why we need this.

SIMBA MOBi: ABM from start to end.



Time required for a 10% Simulation Run of Switzerland (11 million agents):

<1 hour (populationsim)

2020: 6 hours (100%: 24hours)

2022: 6 hours (100%: 24hours)

2025: 6 hours (100%: 24hours)

2019: 52 hours

(QSim, 300 it.)

2022: 24 hours

(Hermes, optimized replanning, 200 it.)

2026: 14 hours

(DSim 200 it.)

The old MOBi Plans in Python (and why we chose to replace it)

- Visum has a long standing @ SBB, many co-developments with PTV
- ABM as a part of Visum since 2020
- Allows nice visualizations of daily plans
- MOBi Plans uses the API and data models, but lives in its own python setup
- Highly optimized, hard to read python code

Company
strategy:
Cloud first.

- Requires a Windows Server with 150 GB+ memory
- Not at all ready for the cloud

n	Aktiv	Verfahren	ugsobjekt	Variante/Datei	Meldungen	Kommentar
	☐	Gruppe Load activities	2			Load activities in the ABM demand
	☐	Skript ausführen		load_activities.py		Load activities in the ABM demand
	☐	Gruppe Load zonal vari	4			Load zonal variables
	☐	Skript ausführen		load_external_zonal_vars.py		Load external zonal information (d
	☐	Gruppe Load synpop d	6 - 10			Load synpop data
	☐	Skript ausführen		load_synpop.py		Load synpop to Visum
	☐	Skript ausführen		adjust_synpop_for_case_study.py		Import additional facilities and fit p
	☐	Skript ausführen		calc_household_compositions.py		Creates household attributes base
	☐	Skript ausführen		create_activity_locations.py		Create activity locations based on
	☐	Skript ausführen		calc_zonal_aggregates.py		Calculate zonal aggregates
	☐	Gruppe Calculate additi	12 - 13			Calculate additional zonal attribute
	☐	Skript ausführen		calc_destination_attractions.py		Calculate zonal destination attract
	☐	Skript ausführen		calc_atimes_and_pcosts.py		Calculate zonal access times and p
	☐	Gruppe Import skims	15 - 17			Import skims
	☐	Skript ausführen		load_car_skims.py		Import car skims
	☐	Skript ausführen		load_pt_skims.py		Import public transport skims
	☐	Skript ausführen		load_kalif_matrices.py		Import location choice calibration
	☐	Gruppe Calculate acces	19 - 20			Calculate accessibilities
	☐	Skript ausführen		prepare_mobi_plans.py		Prepare all variables and paramete
	☐	Skript ausführen		zone_based_accessibilities.py		Calculate zone-based accessibilities
	☐	Gruppe Run ownerhshi	22 - 24			Run ownership model and aggre
	☐	Skript ausführen		population_sampling.py		Population Sampling
	☐	Skript ausführen		ownership_models.py		Make mobility tool ownership choi
	☐	Skript ausführen		process_mobility_tools_and_calc_aggr		Process mobility tool choices and ;
	☒	Gruppe Generate endo	26 - 28			Generate endogenous demand
	☒	Skript ausführen		simulate_full_mobi_plans.py	2272735	Run all mobi-plans stages.
	☒	Skript ausführen		railaccess_choiceset.py	31 Hinwei	Generate person attributes for rail
	☒	Skript ausführen		calc_extra_trip_variables.py		Calculate trip purpose and directio
	☒	Gruppe Export location	30 - 32			Export locations and persons/trips
	☒	Skript ausführen		visum_to_facilities.py		Export locations from Visum to MA
	☒	Skript ausführen		visum_to_plans.py		Export persons and trips from Visu
	☐	Skript ausführen		stats_to_csv.py		Generates CSV tables with the mi
	☒	Gruppe Export zonal inf	34			Export zonal information as shapef
	☒	Skript ausführen		export_mobi_zones_shapefile.py	1 Fehler	Export zonal information as shapef
	☐	Gruppe Generate repor	36 - 37			Generate report.
	☐	Skript ausführen		load_sim_results.py		Imports the results from a MOBi.si
	☐	Skript ausführen		stats_to_csv.py		Generates CSV tables with the mi

Replacement ideas for a Drop-In Replacement

Native python

- Re-use existing code and implementation logic
- Cryptic code remains cryptic
- Python widely used in choice models
- Presumably easier to on-board more people unfamiliar to MATSim
- Something similar has been done in New Zealand (MONTY), so a certain speedup can be expected

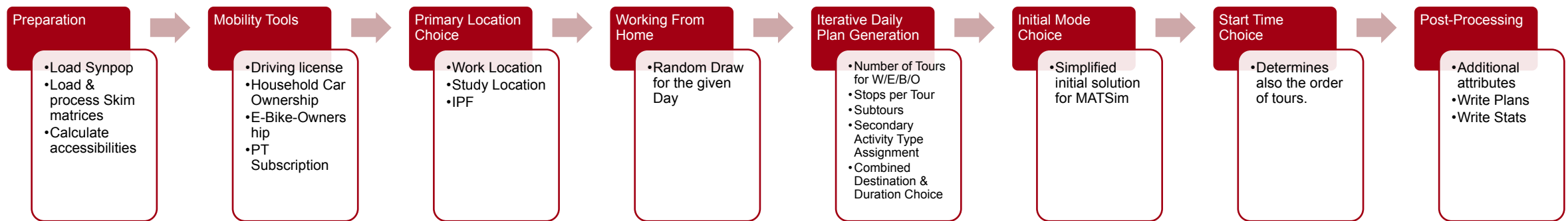
Java Re-Implementation

- Use MATSim API (Persons, Facilities, Households, Plans...)
- Build on existing JAVA & MATSim knowledge
- Start from sketch
- Reduce file I/O
- Automation of the full pipeline in native JAVA code possible
- Uncertain how fast Java Objects are in comparison Pandas Dataframes



2. Into the details.

The model steps.



Implementation: The big picture

- Dependency Injection is your (best) friend
- MOBiPlans Runner determines default order of model steps
- Most models simply loop through the population (in parallel, if useful)

```
private void runEBikeOwnershipModel() { 1 usage  ⤵ Joschka Bischoff
    scenario.getPopulation().getPersons().values().parallelStream()
        .forEach(
            capture of extends Person person -> {
                boolean hasEbike45 = false;
                if (AttributeUtils.getAsInteger(person, SynpopConstants.AGE, defaultValue: -1) > 13) {
                    hasEbike45 = ebikeModel.calculate(person);
                }
                person.getAttributes().putAttribute(SynpopConstants.HAS_EBIKE45, hasEbike45);
            }
        );
    logger.info("Finished e-bike calculation.");
}
```

```
MOBiPlansRunner.java  ⓘ ModelStep.java × ⓘ IterativeDailyPlanGenerationOrchestrator.jav

1  package ch.sbb.matsim.plans.models.steps;
2
3  ⓘ public interface ModelStep extends Runnable {  ⤵ Joschka Bischoff *
4  ⓘ     /**
5  ⓘ         * @return The unique key for this model step, used in the config.
6  ⓘ         */
7  ⓘ     String getName(); 17 implementations  ⤵ Joschka Bischoff
8  ⓘ }

public DefaultMOBiPlansModule(MOBiPlansConfig config) { this.config = config; }

@Override  ⤵ Joschka Bischoff +1
protected void configure() {
    super.configure();
    bind(MOBiPlansConfig.class).toInstance(config);
    bind(Scenario.class).toInstance(scenario);
    //Zones, Matrices and Strata definitions should be here immediately available after scenario creation
    bind(Zones.class).toProvider(MOBiZonesProvider.class).asEagerSingleton();
    bind(Matrices.class).toProvider(MOBiMatricesProvider.class).asEagerSingleton();
    bind(ModeImpedanceFormulaRepository.class).to(ModeImpedanceFormulaRepositoryFromCSV.class).asEagerSingleton();

    //the following are lazy singletons
    bind(TourBuildingModel.class);
    bind(CombinedDestinationAndDurationChoiceModel.class);
    Multibinder<ModelStep> multibinder = Multibinder.newSetBinder(binder(), ModelStep.class);

    multibinder.addBinding().to(SynpopLoader.class);
    multibinder.addBinding().to(ZoneBasedAccessibilities.class);
    multibinder.addBinding().to(MobilityToolOwnershipModel.class);
    multibinder.addBinding().to(PrimaryLocationChoiceModel.class);
    multibinder.addBinding().to(MOBiPlansOutputWriter.class);
    multibinder.addBinding().to(PopulationSampler.class);
    multibinder.addBinding().to(IterativeDailyPlanGenerationOrchestrator.class);
    multibinder.addBinding().to(CalcZonalAttributes.class);
    multibinder.addBinding().to(InitialModeChoiceModel.class);
    multibinder.addBinding().to(StartTimeChoiceModel.class);
    multibinder.addBinding().to(AssignFacilityLocations.class);
    multibinder.addBinding().to(RailAccessChoiceSet.class);
    multibinder.addBinding().to(PedestrianSpeedModel.class);
    multibinder.addBinding().to(WorkFromHomeModel.class);
    multibinder.addBinding().to(StatsWriter.class);
    multibinder.addBinding().to(MergeExogenousPlans.class);
    multibinder.addBinding().to(AddAdditionalTripVariables.class);
}
```


Implementation: Discrete Choice Models

- Most model steps require several discrete choice models, such as for Household Car Availability or the number of work tours per person.
- Unified model definitions in CSV files

```
public interface ChoiceModel { 17 usages 7 implementations ⓘ u229187 +1

    Object calculate(Attributable attributable); 7 implementations ⓘ u229187

    double getProbabilityForAlternative(Attributable attributable, String alternative);

}
```

	C1 ▾	÷	C2 ▾	÷	C3 ▾	÷	C4 ▾	÷	C5 ▾	÷	C6 ▾	÷
1	type		attribute		values		lambda		beta_true		beta_false	
2	constant		<null>		<null>		<null>		-8.712747513		0	
3	piecewise		age		25,50,65		<null>		0.066808565,0.019011442,-0.023195964,-0.103...		0,0,0,0	
4	linear		is_swiss		<null>		<null>		0.884514105		0	
5	condition		language		==1		<null>		0.634205948		0	
5	condition		level_of_employment		<90,>0		<null>		0.190880612		0	
7	condition		household.couple_type		==1		<null>		0.66783354		0	
3	condition		household.couple_type		==2		<null>		0.341246217		0	
7	condition		household.nb_of_cars		>0		<null>		0.436771224		0	
8	linear		zone.accsib_car/1000000		<null>		<null>		3.655009466		0	

Primary Location Choice: logit + IPF for work & education

Assigns each synthetic person a work/school zone and facility: utility-based destination choice with attraction constraints

Group

Strata by shared attraction attribute; persons by home zone → matrix rows

Utility matrix

$U(i,j)$ for all zones with $A_j > 0$; rows built in parallel

Targets

Rows = persons per row; columns = attraction A_j , scaled to $\sum$ rows

IPF (Furness)

Balance column factors on $\exp(U)$ to hit A_j , N iterations

Zone draw

Monte Carlo per person from $P(j|i)$; Random seeded by person ID and model

Facility draw

Within chosen zone, weight = $\max(\text{FTE}, 1)$; stores zone + car distance

Utility (origin zone × stratum → destination zone)

$$U = \text{ASC} \cdot f + \ln(A_j) + \text{logsum}_m$$

- Logsum over mode-impedance formulas (per activity × stratum) on travel and accessibility metrics
- ASC: zone-pair "Kalif" matrix × matrixFactor, scaled by a factor f .

Doubly-constrained via IPF

$$\begin{aligned} w_{ij} &= \exp(U_{ij}) \cdot b_j \\ \text{trips}_{ij} &= r_i \cdot w_{ij} / \sum_j w_{ij} \\ b_j &\leftarrow b_j \cdot A_j / \sum_i \text{trips}_{ij} \end{aligned}$$

- Row constraint: persons per origin
- Column constraint: A_j (rescaled to $\sum$ persons)
- Output $P(j|i) = w_{ij} / \sum_j w_{ij}$: a logit whose aggregate matches A_j
- Strata within one attraction group share the matrix, so they compete for the same capacity

Secondary Tours: destination & duration choice

Samples candidate duration and destination plans per person and picks one that fits the time budgets

Preparation

Blended travel-time matrix, Kalif + logsum caches, attraction arrays

Duration plans

10 draws per person; main activity split around subtours

Destination plans

3 draws; main zone first, then stops in chronological order

Budget Filter

Keep plans within performing / travel budget, else take the minimum

Combine 10x3

Random start offset; first combo within OOH budget wins

Apply / Repeat

Set max duration + assigned zone; flag stays if nothing fits and loops again

Zone draw (per activity, MNL over all zones)

$$\begin{aligned}
 U &= \ln A_j + LS_{oj} + K_{oj} && \text{main / subtour} \\
 U &= \ln A_j + \frac{1}{2}(LS_{oj} + LS_{jt}) + \frac{1}{2}(K_{oj} + K_{jt}) && [\text{stop } o \rightarrow j \rightarrow t] \\
 P(j) &= \text{softmax}(U), \text{ max-shifted}
 \end{aligned}$$

- Logsum LS per activity × stratum, Kalif K per activity, A_j = activity-specific zonal attraction
- Stops are rubberbanded: outbound towards the main zone, inbound towards home
- Legs with logsum ≤ -19 (impossible) are excluded; education uses the fixed edu zone

Plan selection with time budgets

$$\begin{aligned}
 \text{perf}_p &= \sum \text{durations} && (10 \text{ plans}) \\
 \text{tt}_q &= 1.2 \cdot \sum \text{blended TT} && (3 \text{ plans}) \\
 \text{accept: } &\text{perf} \leq B_p, \text{ tt} \leq B_t, \text{ perf} + \text{tt} \leq \text{Booh}
 \end{aligned}$$

- Blended TT: PT if car > 2·PT, car if PT > 2·car, else mean; walk speed below 1.5 km
- Misfits keep NEEDS_DAILY_REPLANNING and run again in the next iteration
- Best-effort plan is applied even if no combination fits the budgets

The gain.

- Overall model runtime (start to finish):

2 hours for 100% population

- Over 10x speedup compared to the Visum-Python implementation
- Results are similar, less some bugs of the old model
- System independent, fully cloud-ready

3. Great.
Now can I use it?



Code availability

- At the time being, code is available via:
<https://github.com/SchweizerischeBundesbahnen/matsim-sbb>

This is a mirror of our internal repository, updated every now and then.

- Setting up a model is highly customized.
 - Current setup is tailored to our needs.
- Mid-term idea: Move generalizable code into MATSim contrib, add an understandable toy model

Wrap up.

- Highly successful transformation
- Around three months full-time work to re-implement the model, stretched over a year
- New model setup gives us a good structure for future model improvements:
 - Improved WfH model
 - Include non-home trips
 - Create a weekend model, or even week-long plans



Questions?

Contact: joschka.bischoff@sbb.ch

